

From the Bloch Sphere to Cosmological Posteriors

A Code-Level Bridge Between Classical and Quantum Bayesian Inference

Andrés García Rivera

Engineering Physics, Universidad Iberoamericana (IBERO)

June 24, 2026

Abstract

This document is the theoretical companion to the *Quantum Algorithms for Cosmology* code-base. It is written for a reader who is at home with cosmology and Bayesian statistics but whose quantum-computing knowledge stops at the Bloch sphere. Its purpose is threefold: to separate and explain every function of the code under three headings (Physics, Statistics, Quantum Computation); to build a bridge from a single qubit to the quantum samplers, so the connection between circuit and inference is explicit; and to give the time and space complexity of each component. The guiding scientific principle throughout is *fidelity, not advantage*: for classical parametric model fitting there is no structural quantum speed-up to be had, so the genuine contribution is to show that quantum methods faithfully *replicate* trusted classical results. We make that idea precise by casting the whole project as a *classical-quantum ablation framework*, in which each algorithmic component is a substitution point that is either FAITHFUL (a quantum re-encoding of the exact classical rule, expected to reproduce it) or ALGORITHMIC (a genuinely different quantum algorithm, expected to differ).

Contents

1	Preliminaries	2
1.1	Purpose and the code→theory map	2
1.2	Notation and the guiding principle	2
2	Physics	2
2.1	The Friedmann function and the model registry	3
2.2	From $E^2(z; \theta)$ to observables	3
3	Statistics	4
3.1	Posterior, likelihoods and the marginalized M_{abs}	4
3.2	Vectorized batch evaluation	4
3.3	Diagnostics: τ , ESS and split- $\hat{R}$	4
4	Quantum Computation: from one qubit to the circuits	5
4.1	One qubit, rotations, and the birth of probability	5
4.2	Amplitude encoding of the Metropolis rule (a FAITHFUL cell)	5
4.3	Registers, superposition and the parameter grid	5
4.4	Entanglement and the hardware-efficient ansatz	6
4.5	The quantum proposal (an ALGORITHMIC cell)	6

5	MCMC and Variational Inference: two ways to know a posterior	6
5.1	MCMC: approximate by <i>sampling</i>	6
5.2	Variational Inference: approximate by <i>optimizing</i>	7
5.3	Why they are fundamentally different	7
5.4	The quantum counterparts, sub-algorithm by sub-algorithm	7
5.5	Why QMCMC 50% and 100% give the same corner plot	8
6	The samplers	9
6.1	QMCMC	9
6.2	QVMC and the adaptive grid	9
6.3	Genetic optimizers (CGA / QGA)	9
7	The classical–quantum ablation framework	9
8	Complexity summary	10

1 Preliminaries

1.1 Purpose and the code→theory map

The codebase is organized so that *physics*, *statistics*, and *sampling/optimization* never bleed into one another. All physics lives in `cosmo_core.py`; the samplers in `cosmo_modular_quantum.py` (simulator) and `qpu_cosmo_samplers.py` (hardware); the global optimizers in `cosmo_genetic_optimizers.py`. This document mirrors that separation. Table 1 is the master index: each function appears under exactly one category, with the section that explains it and its complexity.

Function / object	Category	Section	Complexity (time)
<code>CosmoModel, _E2_*</code>	Physics	§2	$\mathcal{O}(N_z)$
<code>Posterior._mu_theory</code>	Physics	§2	$\mathcal{O}(N_z)$
<code>Posterior.log_prob_batch</code>	Statistics	§3	$\mathcal{O}(B(N_z + N_{\text{SN}}^2))$
<code>chi2_pantheon_plus</code>	Statistics	§3	$\mathcal{O}(N_{\text{SN}}^2)$
<code>autocorr_time_fft</code>	Statistics	§3	$\mathcal{O}(N \log N)$
<code>split_rhat</code>	Statistics	§3	$\mathcal{O}(MN)$
<code>QMCMCModular</code>	Statistics/QC	§6.1	per step $\mathcal{O}(M \cdot c_{\text{post}})$
<code>hadamard_accept_log_batch</code>	QC (FAITHFUL)	§4	$\mathcal{O}(M)$
<code>QuantumProposalEngine</code>	QC (ALGORITHMIC)	§4	$\mathcal{O}(2^{n_q})/\text{refill}$
<code>QVMCModular</code>	Statistics/QC	§6.2	target $\mathcal{O}(2^{n_{\text{qpp}}})$
<code>estimate_grid_window</code>	Statistics	§6.2	$\mathcal{O}(n_{\text{steps}} M c_{\text{post}})$
<code>CGA/QGA</code>	Statistics/QC	§6.3	$\mathcal{O}(G \cdot P \cdot d)$
<code>compute_quantumness</code>	Framework	§7	$\mathcal{O}(1)$

Table 1: Master map: every component, its category, the section that explains it, and its dominant time complexity. B = batch size, N_z = distance-grid size, N_{SN} = number of supernovae, M = chains, N = chain length, d = parameters, n_{qpp} = qubits per parameter, G = generations, P = population, c_{post} = cost of one posterior evaluation.

1.2 Notation and the guiding principle

We write the dimensionless Hubble function as $E(z; \theta)$ with $E^2(z; \theta) = H^2(z; \theta)/H_0^2$, and θ for the parameter vector (by convention $\theta_0 = \Omega_m$, $\theta_1 = H_0$). Quantum states use Dirac notation; a single qubit is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$.

Remark 1.1 (Fidelity, not advantage). *For fitting a handful of cosmological parameters to fixed data, the bottleneck is the likelihood evaluation, which is classical arithmetic over the data. No quantum subroutine in this project changes that asymptotics, and we make no claim of quantum advantage. The scientific value is the demonstration that quantum re-encodings of the inference steps reproduce the classical posterior — a fidelity result. Section 7 turns this into a falsifiable framework.*

2 Physics

The only physics a model must expose is $E^2(z; \theta)$; every observable is derived generically from it. This is the `CosmoModel` contract.

2.1 The Friedmann function and the model registry

Definition 2.1 (Background model). *A flat FLRW background with matter, radiation and dark energy obeys*

$$E^2(z; \theta) = \Omega_m(1+z)^3 + \Omega_r(1+z)^4 + \Omega_{\text{DE}} f_{\text{DE}}(z), \quad (1)$$

with $\Omega_{\text{DE}} = 1 - \Omega_m - \Omega_r$ (flatness) and $f_{\text{DE}}(z)$ the dark-energy density evolution, $f_{\text{DE}}(0) = 1$. The Hubble rate is $H(z; \theta) = H_0 E(z; \theta)$.

The five registered models differ only in f_{DE} :

$$\Lambda\text{CDM: } f_{\text{DE}} = 1, \quad (2)$$

$$\text{wCDM: } f_{\text{DE}} = (1+z)^{3(1+w)}, \quad (3)$$

$$\text{CPL: } f_{\text{DE}} = (1+z)^{3(1+w_0+w_a)} \exp(-3w_a z/(1+z)), \quad (4)$$

$$\text{PEDE: } f_{\text{DE}} = 1 - \tanh(\log_{10}(1+z)), \quad (5)$$

$$\text{GEDE: } f_{\text{DE}} = \frac{1 - \tanh(\Delta \log_{10} \frac{1+z}{1+z_t})}{1 + \tanh(\Delta \log_{10}(1+z_t))}, \quad (1+z_t) = \left(\frac{1-\Omega_m}{\Omega_m}\right)^{1/3}. \quad (6)$$

Proposition 2.1 (Emergent dark-energy normalization). *PEDE and GEDE satisfy $f_{\text{DE}}(0) = 1$ exactly, hence $E^2(z; \theta)|_{z=0} = 1$.*

Proof. For PEDE, $\tanh(\log_{10} 1) = \tanh 0 = 0$, so $f_{\text{DE}}(0) = 1$. For GEDE at $z = 0$, $\frac{1+z}{1+z_t} = \frac{1}{1+z_t}$, so the numerator is $1 - \tanh(-\Delta \log_{10}(1+z_t)) = 1 + \tanh(\Delta \log_{10}(1+z_t))$, identical to the denominator; their ratio is 1. This holds for every Ω_m, Δ and is asserted in `tests/test_core.py::test_emergent_de_normalization`. $\square$

Remark 2.1 (Convention caveat checked at $z \neq 0$). *An inverted sign of z_t would also give $f_{\text{DE}}(0) = 1$ but the opposite high- z evolution. The test `test_gede_reduces_to_lcdm_at_delta_zero` therefore checks $\Delta = 0 \Rightarrow f_{\text{DE}} \equiv 1$ at $z = 0.5, 1, 2$, not only at $z = 0$.*

2.2 From $E^2(z; \theta)$ to observables

Two observables enter the likelihood. The Hubble rate $H(z; \theta)$ is read directly. The distance modulus requires the comoving distance:

$$d_C(z) = \frac{c}{H_0} \int_0^z \frac{dz'}{E(z; \theta)[z']}, \quad d_L = (1+z) d_C, \quad \mu_{\text{th}} = 5 \log_{10}(d_L/\text{Mpc}) + 25. \quad (7)$$

Remark 2.2 (Flatness, made explicit). *The code takes the transverse comoving distance equal to d_C , valid only for $\Omega_k = 0$. All five models are flat. A curved extension would need $d_M = (c/H_0 \sqrt{|\Omega_k|}) \text{sinn}(\sqrt{|\Omega_k|} H_0 d_C/c)$; `_mu_theory` documents this so a future non-flat model is not silently wrong.*

Complexity. The integral is computed once on a fixed grid of N_z points by cumulative trapezoid, then interpolated to the supernova redshifts: $\mathcal{O}(N_z)$ time and space, and crucially *model-agnostic* — the same code serves CPL or GEDE. The earlier point-by-point `scipy.integrate.quad` was $\mathcal{O}(N_{\text{SN}} \cdot (\text{integrand evals}))$ and $\sim 10\times$ slower.

3 Statistics

3.1 Posterior, likelihoods and the marginalized M_{abs}

The log-posterior ties model, data and prior:

$$\log P(\theta \mid \text{data}) = \log \pi(\theta) - \frac{1}{2} \chi^2(\theta), \quad \chi^2 = \chi_{\text{CC}}^2 + \chi_{\text{SN}}^2. \quad (8)$$

The $H(z)$ (CC+BAO) term is diagonal, $\chi_{\text{CC}}^2 = \sum_i ((H_{\text{obs},i} - H(z_i; \theta))/\sigma_i)^2$. The supernova term marginalizes the unknown absolute magnitude M_{abs} analytically.

Proposition 3.1 (Analytic M_{abs} marginalization, Goliath 2001). *With residuals $\Delta_i = m_{\text{obs},i} - \mu_{\text{th}}(z_i; \theta)$ and a flat prior on the constant offset M_{abs} , integrating it out gives*

$$\chi_{\text{eff}}^2 = A - \frac{B^2}{C}, \quad A = \Delta^\top C_*^{-1} \Delta, \quad B = \Delta^\top C_*^{-1} \mathbf{1}, \quad C = \mathbf{1}^\top C_*^{-1} \mathbf{1}, \quad (9)$$

with C_* the (diagonal for Pantheon 2018, full for Pantheon+) covariance. The result is invariant under $m_{\text{obs}} \rightarrow m_{\text{obs}} + \text{const}$.

The shift-invariance is the defining correctness property and is asserted in `test_mabs_marginalization_shift_invariance`.

Remark 3.1 (Combined datasets and H_0). *SNe with M_{abs} marginalized constrain only the shape $E(z; \theta)$, not H_0 . When CC+BAO is added, H_0 is anchored directly by the $H(z)$ term: the combined fit does not marginalize H_0 , only the SNe nuisance offset. This is intentional — CC+BAO is what pins H_0 — and is documented in *Posterior.chi2*.*

3.2 Vectorized batch evaluation

`log_prob_batch` evaluates the posterior for a batch of B parameter vectors in one pass, broadcasting $E^2(z; \theta)$ over the redshift grid. This is the single hot path for the QVMC grid target and the vectorized QMCMC kernel.

Complexity. For a batch of B : the CC term is $\mathcal{O}(BN_{\text{CC}})$; the SNe distance integral is $\mathcal{O}(BN_z)$; the Pantheon+ quadratic form $\Delta^\top C^{-1} \Delta$ is $\mathcal{O}(BN_{\text{SN}}^2)$ (the C^{-1} is precomputed once). Time $\mathcal{O}(B(N_z + N_{\text{SN}}^2))$; space $\mathcal{O}(BN_z + N_{\text{SN}}^2)$.

3.3 Diagnostics: τ , ESS and split- $\hat{R}$

Definition 3.1 (Integrated autocorrelation time). *For a stationary chain with normalized autocorrelation $\rho(t)$, $\tau = 1 + 2 \sum_{t \geq 1} \rho(t)$. On a finite chain the sum must be truncated; `autocorr_time_fft` uses **Sokal automatic windowing**: accumulate $\tau(M) = 1 + 2 \sum_{t=1}^M \rho(t)$ and stop at the first M with $M \geq c\tau(M)$ ($c = 5$). The autocovariance comes from one zero-padded FFT, so the cost is $\mathcal{O}(N \log N)$.*

Remark 3.2 (Why the old cutoff was wrong). *The previous rule “first lag with $\rho < 0.05$, else $N/4$ ” returned the same sentinel ($\arg \max = 0$) for a chain that never decorrelates and for one already decorrelated at lag 1, conflating the worst and best chains. The Sokal rule is self-consistent and is the community standard (emcee). Tests: `test_autocorr_white_noise` ($\tau \approx 1$), `test_autocorr_ar1` ($AR(1) \phi = 0.9 \Rightarrow \tau \approx 19$), and the regression `test_autocorr_excellent_chain_regression`.*

The effective sample size is $\text{ESS} = MN/\tau_{\max}$ with $\tau_{\max}$ the worst per-chain-averaged τ across parameters (the slowest direction governs independence). For weighted QVMC samples the Kish ESS $(\sum w)^2/\sum w^2$ is used instead.

Definition 3.2 (Rank-normalized split- $\hat{R}$, Vehtari et al. 2021). *Split each chain in half (catching within-chain drift), rank-normalize the pooled draws (robust to heavy tails), then form $\hat{R} = \sqrt{(\frac{N-1}{N}W + B/N)/W}$ from the between- (B) and within-chain (W) variances; report the max over parameters. Convergence threshold $\hat{R} < 1.01$.*

Remark 3.3 (Why 1.01 matters here). *The project’s central claim is “quantum reproduces classical.” Agreement between two unconverged chains proves nothing, so a lax threshold (1.05) would make the comparison vacuous. The stricter rank-normalized split- $\hat{R} < 1.01$ guards this; tests `test_split_rhat_converged_vs_drift` and `test_split_rhat_catches_within_chain_trend` show it separates converged from drifting chains and catches a trend the classical $\hat{R}$ misses.*

4 Quantum Computation: from one qubit to the circuits

This is the bridge. We start where the reader is — one qubit on the Bloch sphere — and build up exactly the machinery the samplers use.

4.1 One qubit, rotations, and the birth of probability

A qubit state $|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}|1\rangle$ is a point (θ, φ) on the Bloch sphere. The rotation $R_Y(\lambda)$ takes $|0\rangle$ to $\cos\frac{\lambda}{2}|0\rangle + \sin\frac{\lambda}{2}|1\rangle$, so measuring in the computational basis gives

$$P(|1\rangle) = \sin^2(\lambda/2), \quad P(|0\rangle) = \cos^2(\lambda/2). \quad (10)$$

Equation (10) is the entire trick behind the FAITHFUL acceptance cell below: any target probability $A \in [0, 1]$ is obtained with $\lambda = 2 \arccos \sqrt{A}$.

4.2 Amplitude encoding of the Metropolis rule (a FAITHFUL cell)

Proposition 4.1 (Quantum acceptance reproduces Metropolis exactly). *Let $\Delta = \log P(\theta') - \log P(\theta)$ and $A = \min(1, e^\Delta)$ the Metropolis acceptance. Preparing $R_Y(2 \arccos \sqrt{A})|0\rangle$ and reading $|\langle 0|\psi\rangle|^2$ returns A exactly. Hence a chain using the quantum acceptance is identical (given the same proposal and random stream) to the classical one.*

This is why the acceptance is labelled FAITHFUL: the circuit computes the *same number* the CPU would. `hadamard_accept_log_batch` evaluates all M chains’ single-qubit circuits in one Aer job; the test `test_qpu_helpers::test_b1_metropolis_rule` checks monotonicity in Δ (the regression guard for the historically inverted readout).



4.3 Registers, superposition and the parameter grid

Stacking n qubits and applying $H^{\otimes n}$ creates a uniform superposition over 2^n basis states. The QVMC/QGA encode a discretized parameter grid in such a register: parameter i occupies `nqpp` qubits, so d parameters need $d \cdot \text{nqpp}$ qubits and the grid has $2^{d \cdot \text{nqpp}}$ points. `_build_theta_table` is the bijection between a basis-state index and a θ on this grid.

4.4 Entanglement and the hardware-efficient ansatz

Two-qubit gates (CX, controlled- R_Y) create entanglement — correlations no product state can represent. The QVMC *ansatz* interleaves single-qubit R_Y/R_Z rotations with a ring of CX gates; its tunable angles φ define a variational family $|\psi(\varphi)\rangle$ whose measurement distribution $|\langle x|\psi(\varphi)\rangle|^2$ is fitted to the posterior.

4.5 The quantum proposal (an ALGORITHMIC cell)

The QMCMC proposal circuit (Sarracino et al. 2025) is $H^{\otimes n} \rightarrow [R_Y R_Z + \text{chained } C R_Y]^{\times L} \rightarrow H^{\otimes n} \rightarrow R_Y$; the real parts of its output amplitudes define a displacement of θ . Because the displacement depends only on random angles (not on the current θ), a whole block is generated in one Aer job and consumed from a queue (`QuantumProposalEngine`). The raw displacement is zero-mean but has std ≈ 0.35 ; it is rescaled to unit std so it is a drop-in for the classical $\mathcal{N}(0, 1)$ proposal (keeping acceptance in a healthy band). This is an ALGORITHMIC cell: it is genuinely not the classical Gaussian, so it changes the chain.

Complexity. A statevector refill of b proposals on n_q qubits is $\mathcal{O}(b \cdot 2^{n_q})$ time, $\mathcal{O}(2^{n_q})$ space; amortized per step it is one queue pop, $\mathcal{O}(1)$.

5 MCMC and Variational Inference: two ways to know a posterior

Both samplers attack the same object — the posterior $P(\theta \mid \text{data})$ — but in fundamentally different ways. Understanding why they differ is the key to reading the results, and to understanding why some quantum substitutions change the answer and others cannot.

5.1 MCMC: approximate by *sampling*

Markov Chain Monte Carlo does not try to write down P . It builds a random walk through parameter space whose *visited points*, after equilibration, are distributed according to P . The Metropolis–Hastings recipe: from the current θ_t , draw a proposal $\theta' \sim q(\cdot \mid \theta_t)$, and accept it with probability

$$A(\theta_t \rightarrow \theta') = \min\left(1, \frac{P(\theta') q(\theta_t \mid \theta')}{P(\theta_t) q(\theta' \mid \theta_t)}\right). \quad (11)$$

For a symmetric proposal ($q(\theta' \mid \theta_t) = q(\theta_t \mid \theta')$, e.g. a Gaussian step) the ratio collapses to $A = \min(1, P(\theta')/P(\theta_t)) = \min(1, e^\Delta)$ with $\Delta = \log P(\theta') - \log P(\theta_t)$. Accepted and rejected steps both produce a sample (on rejection $\theta_{t+1} = \theta_t$).

Proposition 5.1 (Why it works). *The Metropolis rule satisfies detailed balance, $P(\theta) T(\theta \rightarrow \theta') = P(\theta') T(\theta' \rightarrow \theta)$, where T is the full transition kernel (proposal $\times$ acceptance). Detailed balance makes P the stationary distribution of the chain, so for large t the samples $\{\theta_t\}$ are draws from P — regardless of the proposal q , which affects only the efficiency of mixing, not the destination.*

The two sub-algorithms of MCMC are therefore:

- **proposal** — where to step (q); changes how fast you explore, not where you end up;
- **acceptance** — whether to keep the step (Eq. 11); enforces detailed balance and fixes the stationary distribution.

The output is a set of correlated samples; means, credible intervals and the corner plot are computed by histogramming them. Diagnostics (τ , ESS, split- $\hat{R}$, §3) ask “has the walk equilibrated and how many *independent* samples do I effectively have?”

5.2 Variational Inference: approximate by *optimizing*

VI takes the opposite stance. Instead of sampling, it picks a family of simple distributions $Q_\varphi(\theta)$ and *tunes* φ until Q_φ is as close as possible to P , measured by the Kullback–Leibler divergence

$$\text{KL}(Q_\varphi \| P) = \int Q_\varphi(\theta) \log \frac{Q_\varphi(\theta)}{P(\theta | \text{data})} d\theta = \mathbb{E}_{Q_\varphi}[\log Q_\varphi - \log P]. \quad (12)$$

Inference becomes *optimization*: $\varphi^\star = \arg \min_\varphi \text{KL}(Q_\varphi \| P)$, solved by gradient descent (classical VI here uses a grid representation of Q_φ and COBYLA/SGD). Once trained, $Q_{\varphi^\star}$ is the approximate posterior — you have a closed object you can sample or read moments from directly, with no autocorrelation and no burn-in.

In this project Q_φ lives on the discrete 2^{nqpp} grid per parameter (§4), so the two sub-algorithms of VI are:

- **training** — minimize Eq. 12 over φ ; this is where the approximation is actually fitted;
- **sampling** — once trained, draw from $Q_{\varphi^\star}$ to report means and a corner plot;
- **normalization** — ensure Q_φ sums to one over the (masked) grid support, so the KL and the reported distribution are proper.

5.3 Why they are fundamentally different

	MCMC	Variational Inference
Strategy	sample the posterior	optimize a proxy to the posterior
Output	correlated draws	a fitted closed-form $Q_{\varphi^\star}$
Exactness	asymptotically exact (given mixing)	only as good as the family Q_φ
Failure mode	slow mixing / non-convergence	too-rigid family, biased Q
Cost driver	$\# \text{steps} \times \text{likelihood}$	$\# \text{iters} \times \text{grid } 2^{n \cdot \text{nqpp}}$
Diagnostic	$\hat{R}$, ESS, τ	KL curve vs iteration

Table 2: MCMC is unbiased but can be slow to converge; VI is fast and gives a closed object but is only as good as the chosen family Q_φ can represent. They are complementary, not interchangeable — which is why the project runs both and compares them against the *same* classical baselines.

A blunt way to put it: MCMC is asymptotically exact but you never quite know when you have arrived; VI always “arrives” (the optimizer converges) but to a point that is only as correct as your family Q_φ allows. The KL floor you see in the QVMC training curve (it bottoms out at some $\text{KL} > 0$) is exactly this family-limitation: even a perfectly trained Q_φ on a coarse grid cannot match P to zero KL.

5.4 The quantum counterparts, sub-algorithm by sub-algorithm

Each classical sub-algorithm has a quantum re-encoding. The crucial point — and the heart of the ablation framework (§7) — is that some re-encodings compute the *identical* mathematical object (FAITHFUL), while others are genuinely different algorithms (ALGORITHMIC).

Sub-alg.	Classical	Quantum	Kind
MCMC proposal	Gaussian step $\theta' = \theta + \mathcal{N}(0, \sigma)$	Statevector circuit (Sarracino): random-angle circuit whose amplitudes define the displacement, rescaled to unit std	ALGORITHMIC
MCMC acceptance	$A = \min(1, e^\Delta)$ on the CPU	$R_Y(2 \arccos \sqrt{A}) 0\rangle$, read $P(0\rangle) = A$ — the <i>same</i> number	FAITHFUL
VI training	gradient descent on KL (COBYLA/SGD)	parameter-shift gradients of a variational circuit (or SPSA on hardware) — a different optimizer reaching a different optimum	ALGORITHMIC
VI sampling	draw indices from the trained grid pmf	measure shots of the trained state $ \psi(\varphi^*) ^2$ — same distribution, finite-shot noise	FAITHFUL
VI normalization	divide by the sum over support	exact renormalization over the masked support — identical	FAITHFUL

5.5 Why QMCMC 50% and 100% give the same corner plot

This is the question the corner plots provoke, and it has a clean answer.

Recall the QMCMC ladder switches its two sub-algorithms to quantum one at a time:

$$\underbrace{0\%}_{\text{classical}} \rightarrow \underbrace{50\%}_{+ \text{ quantum } \textit{proposal}} \rightarrow \underbrace{100\%}_{+ \text{ quantum } \textit{acceptance}}.$$

The step from 50% to 100% switches **only the acceptance** from the CPU formula $\min(1, e^\Delta)$ to its quantum encoding $R_Y(2 \arccos \sqrt{A}) |0\rangle$. But by the proposition of §4.2, that circuit returns *exactly* $A = \min(1, e^\Delta)$ — the same number, to machine precision. The acceptance is a FAITHFUL cell.

Now recall *why* the acceptance cannot change the answer even in principle: by detailed balance, the acceptance rule *defines the stationary distribution*, and both 50% and 100% use the *same* acceptance rule (Metropolis), computed two different ways that agree bit-for-bit. With the same proposal, the same rule, and the same random-number stream, the two chains take *identical* steps. Their samples are identical, so their corner plots, means and credible intervals coincide exactly.

Contrast the 0% → 50% step, which switches the *proposal* (ALGORITHMIC): the quantum proposal is a genuinely different q , so the chain explores differently and the corner plot shifts slightly (same destination P , different path and mixing). And contrast the QVMC ladder, where the 33% → 67% step switches *training* (ALGORITHMIC) and the KL drops sharply — there the quantum sub-algorithm really is a different optimizer.

Remark 5.1 (The take-away). *A coincident corner plot between two adjacent rungs is not a bug and not a boring null result: it is a measurement. It confirms that the quantum re-encoding of that sub-algorithm is faithful — it reproduces the classical computation exactly. That confirmation is the whole point of the project’s “fidelity, not advantage” thesis. The rungs that should coincide do, and the rungs that should differ do.*

6 The samplers

6.1 QMCMC

`QMCMCModular` is a hand-written Metropolis–Hastings (not emcee), so that each component can be swapped classical $\leftrightarrow$ quantum and the baseline and quantum run share the exact transition structure, step scale and RNG stream. The kernel is vectorized: one `log_prob_batch` call scores all M chains per step.

Complexity. Per step the cost is dominated by the likelihood, $\mathcal{O}(M c_{\text{post}})$ with $c_{\text{post}} = \mathcal{O}(N_z + N_{\text{SN}}^2)$; the quantum proposal adds $\mathcal{O}(M)$ queue pops and the quantum acceptance one $\mathcal{O}(M)$ Aer job. Space $\mathcal{O}(MNd)$ for the stored chains.

6.2 QVMC and the adaptive grid

`QVMCModular` represents the posterior on the discrete grid as $|\psi(\varphi)|^2$ and minimizes $\text{KL}(Q_\varphi \| P)$. A naive grid over the full `sample_box` collapses the (narrow) cosmological posterior onto 1–3 cells; `estimate_grid_window` (now in `cosmo_core`, shared with the QPU path) runs a short classical pre-fit and centres a zoomed $\pm k\sigma$ window on the posterior, using the *median* (robust to asymmetric posteriors).

Training is the ALGORITHMIC cell: quantum parameter-shift gradients (or SPSA on hardware) reach a different KL minimum than classical COBYLA. Sampling and normalization are FAITHFUL cells (shots from the trained state; exact renormalization).

Complexity. Building the target evaluates the posterior on all $2^{d_{\text{nqpp}}}$ grid points: $\mathcal{O}(2^{d_{\text{nqpp}}} c_{\text{post}})$ time and $\mathcal{O}(2^{d_{\text{nqpp}}})$ space — the **exponential** cost in d_{nqpp} that the `-max-qubits` guard protects. Parameter-shift adds $2n_\varphi$ circuit evaluations per iteration, batched into one Aer job.

6.3 Genetic optimizers (CGA / QGA)

The genetic fitness is the *same* log-posterior, so maximizing fitness $\equiv$ finding the MAP. CGA is pure NumPy; QGA substitutes quantum operators (init / mutation / crossover), all ALGORITHMIC.

Remark 6.1 (The transpile-once fix, A1). *Each QGA operator circuit is parametrized and transpiled once; an individual’s gene bits enter only through bound rotation angles (a bit b with flip probability p maps to $R_Y(2 \arcsin \sqrt{b(1-p) + (1-b)p})$). The hot loop then only `assign_parameters` on the cached transpiled template, instead of building and transpiling one fresh circuit per individual per generation ($P \cdot d$ transpilation/generation in the old code).*

Complexity. $\mathcal{O}(G \cdot P \cdot d)$ classical work per run; the QGA adds one batched Aer job per operator per generation (no per-individual transpilation after A1).

7 The classical–quantum ablation framework

Definition 7.1 (Ablation index, uniform weighting). *The quantumness of a configuration is the fraction of substitution points switched to quantum, every point weighted equally:*

$$\text{index} = 100 \cdot \frac{\#\{\text{quantum components}\}}{\#\{\text{components}\}}. \quad (13)$$

It is an **ablation index**, not a quantum-resource metric: it counts how many algorithmic steps are quantum, nothing more.

Each substitution point is FAITHFUL (a quantum re-encoding of the exact classical rule; a null cell whose job is to verify the substitution introduces no bias) or ALGORITHMIC (a genuinely different quantum algorithm; a treatment cell). The per-method ladders (uniform spacing) are

QMCMC: $0 \rightarrow 50 \rightarrow 100\%$, QVMC: $0 \rightarrow 33 \rightarrow 67 \rightarrow 100\%$, QGA: $0 \rightarrow 33 \rightarrow 67 \rightarrow 100\%$.

Sampler	Substitution point	Kind	Expected outcome of flipping it
QMCMC	proposal	ALGORITHMIC	changes (genuine quantum proposal)
QMCMC	acceptance	FAITHFUL	identical (reproduces Metropolis)
QVMC	sampling	FAITHFUL	$\approx$ identical (shot noise only)
QVMC	training	ALGORITHMIC	changes strongly (lower KL)
QVMC	normalization	FAITHFUL	$\approx$ identical (faithful renorm.)

Table 4: The ablation map. Faithful cells are falsifiable correctness tests; algorithmic cells are where the science happens.

Remark 7.1 (Why this resolves the “ornamental circuit” objection). *A faithful cell runs a quantum circuit that, by construction, returns the same number as the CPU. Far from being ornamental, it is the control of the experiment: its coincidence with the classical baseline is the evidence that the quantum re-encoding is faithful. The framework thus turns the project’s narrative (“some rungs coincide by design, others differ”) into first-class, testable metadata. The historical hand-picked weights are retained only as `legacy_weighted_index` for CSV continuity.*

8 Complexity summary

Component	Time	Space
$E^2(z; \theta), H(z)$	$\mathcal{O}(N_z)$	$\mathcal{O}(N_z)$
μ_{th} (cumulative trapezoid)	$\mathcal{O}(N_z)$	$\mathcal{O}(N_z)$
<code>log_prob_batch</code>	$\mathcal{O}(B(N_z + N_{\text{SN}}^2))$	$\mathcal{O}(BN_z + N_{\text{SN}}^2)$
τ (FFT, Sokal)	$\mathcal{O}(N \log N)$	$\mathcal{O}(N)$
split- $\hat{R}$	$\mathcal{O}(MN)$	$\mathcal{O}(MN)$
QMCMC (per step)	$\mathcal{O}(M c_{\text{post}})$	$\mathcal{O}(MNd)$
quantum proposal (refill)	$\mathcal{O}(b 2^{n_q})$	$\mathcal{O}(2^{n_q})$
quantum acceptance (batch)	$\mathcal{O}(M)$	$\mathcal{O}(M)$
QVMC target build	$\mathcal{O}(2^{d \cdot \text{nqpp}} c_{\text{post}})$	$\mathcal{O}(2^{d \cdot \text{nqpp}})$
QVMC parameter-shift (iter)	$\mathcal{O}(n_\varphi 2^{d \cdot \text{nqpp}})$	$\mathcal{O}(2^{d \cdot \text{nqpp}})$
CGA / QGA (per run)	$\mathcal{O}(GPd)$	$\mathcal{O}(Pd)$

The single dominant scaling to internalize is the QVMC/QGA grid: $2^{d \cdot \text{nqpp}}$ in both time and space. This is the exponential wall that `-max-qubits` guards and the reason a QPU — which never materializes the statevector — is the only way past it for large grids, though at the cost of shot noise and queue-dominated wall time rather than RAM.

Acknowledgements

This document accompanies the *Quantum Algorithms for Cosmology* codebase. The scientific framing throughout is fidelity, not advantage.